



面向目标机器的代码优化

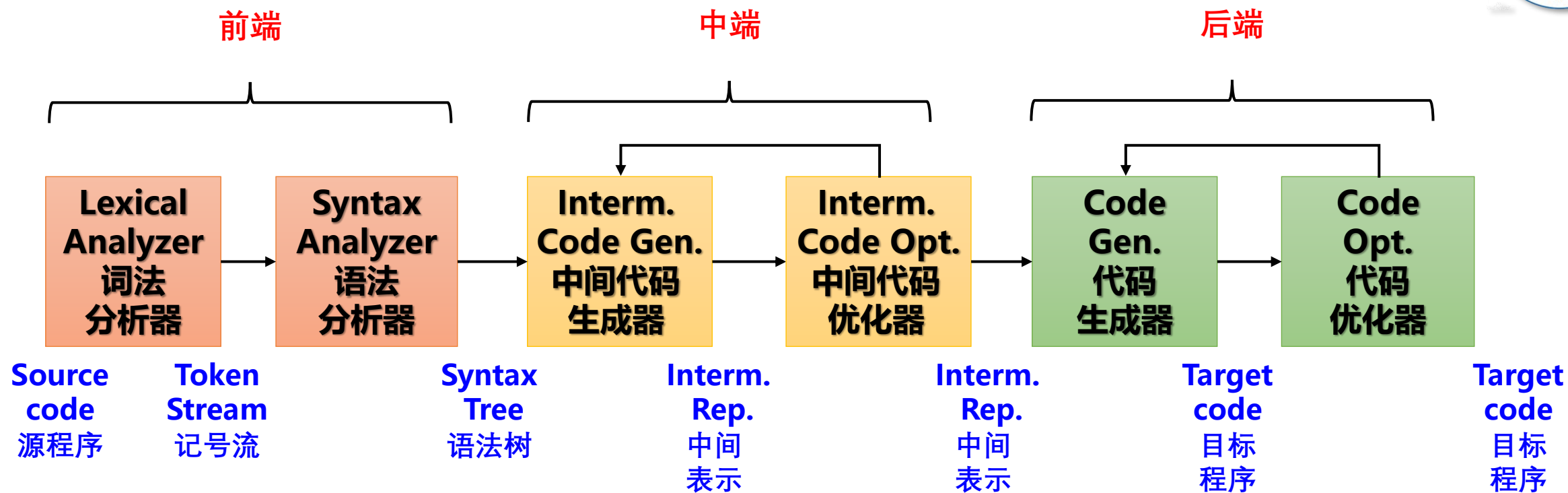
——指令并行与调度

李 诚

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

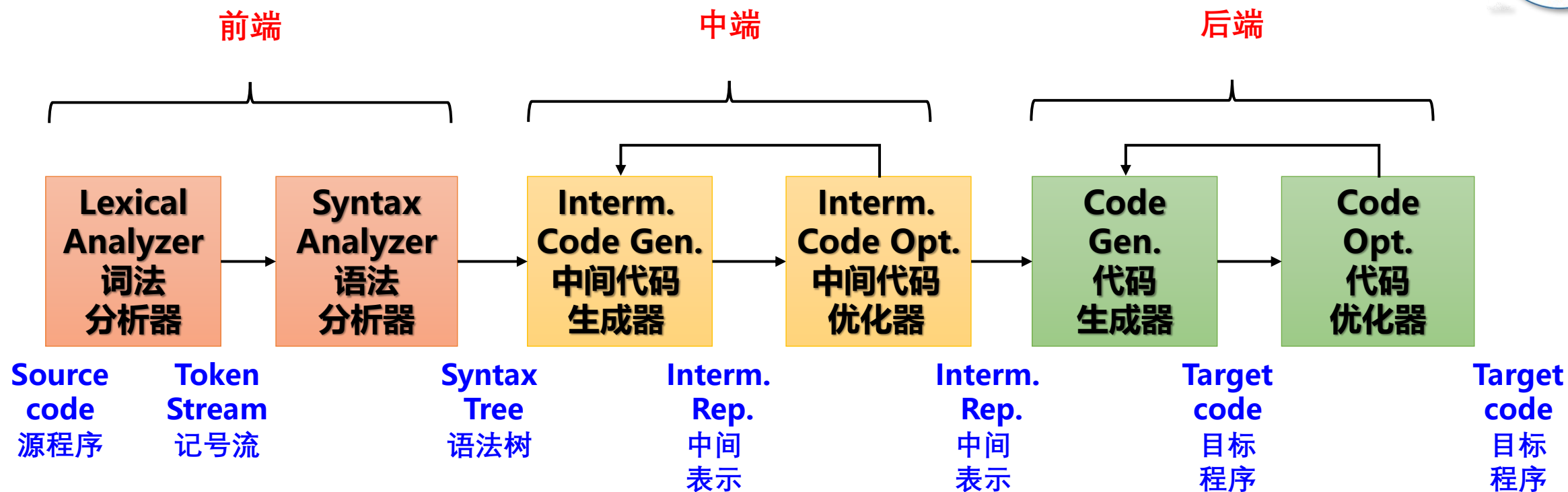
计算机科学与技术学院

2025年11月17日



编译器的基本步骤

前情回顾



技术方案

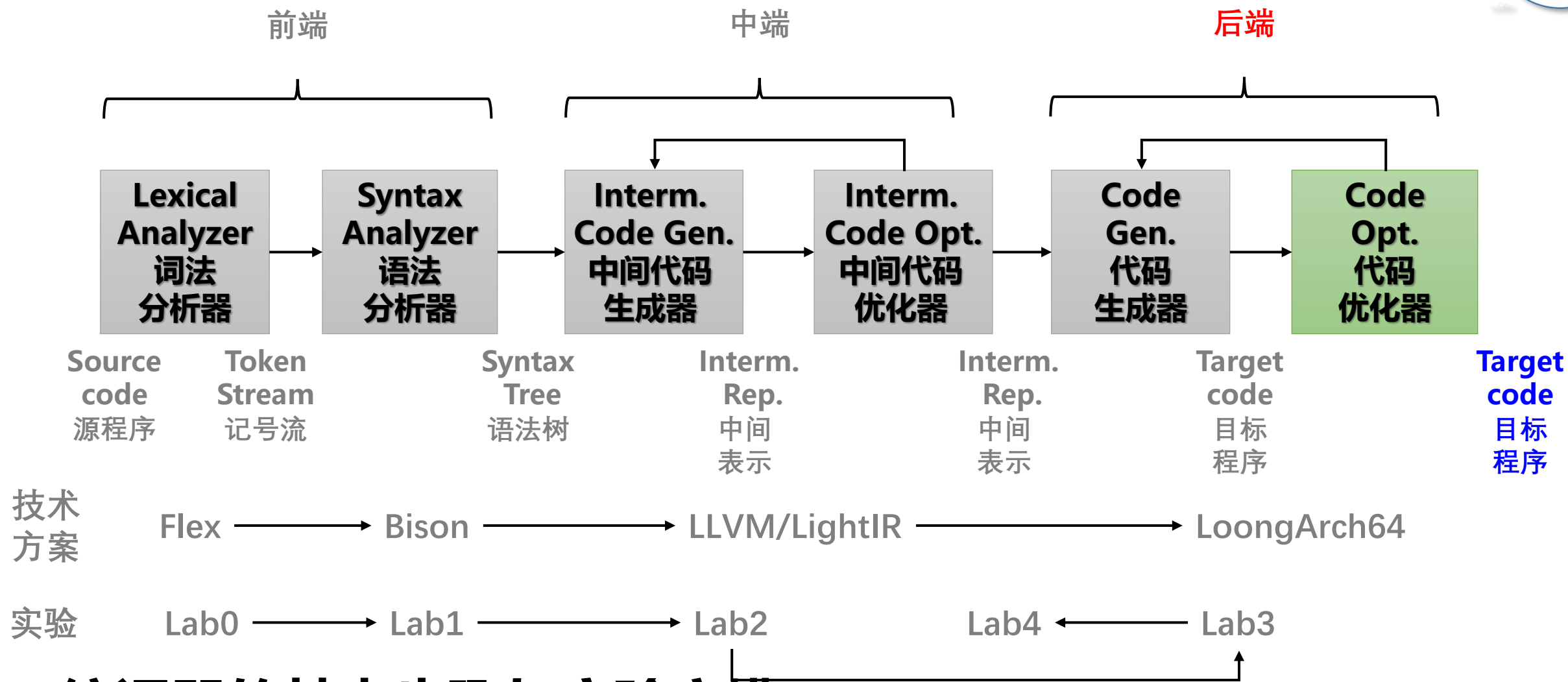
Flex → Bison → LLVM/LightIR → LoongArch64

实验

Lab0 → Lab1 → Lab2 → Lab4 ← Lab3

编译器的基本步骤与实验安排

前情回顾



编译器的基本步骤与实验安排

- ❑ **目标：优化生成的机器代码，与机器无关的优化不同，这一层级的信息是IR层无法获取的。**
- ❑ **面向目标机器的代码优化十分重要，但往往很难实现：**
 - 难以跨机器架构复用
 - 难以跨语言复用

面向目标机器的代码优化 – 种类



❑ 减少操作数量

执行时间的计算公式：

$\text{Execution time} = \text{Operation count} * \text{Machine cycles per operation}$

- 算术操作、内存访问等

❑ 用代价小的操作替换代价高的操作

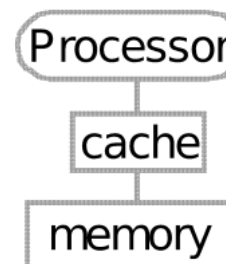
- 例如：4-cycle 乘法 与 1-cycle 移位运算

❑ 降低缓存缺失 (Cache miss)

- 覆盖数据和指令的访问

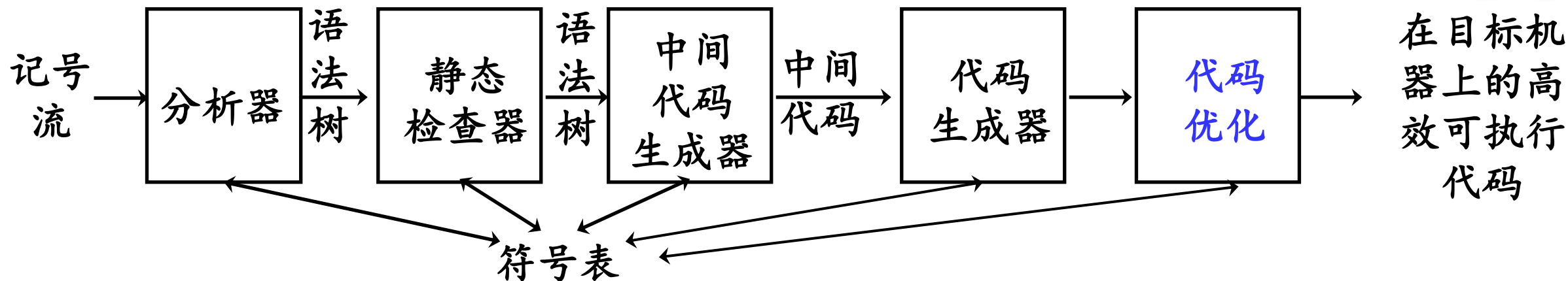
❑ 并行计算

- 单线程内部的指令调度
- 跨线程的并行执行



冯诺依曼体系结构

本节提纲



- ❑ 现代处理器架构
- ❑ 流水线并行的例子
- ❑ 指令调度与数据依赖分析
- ❑ 数据依赖指导下的指令调度

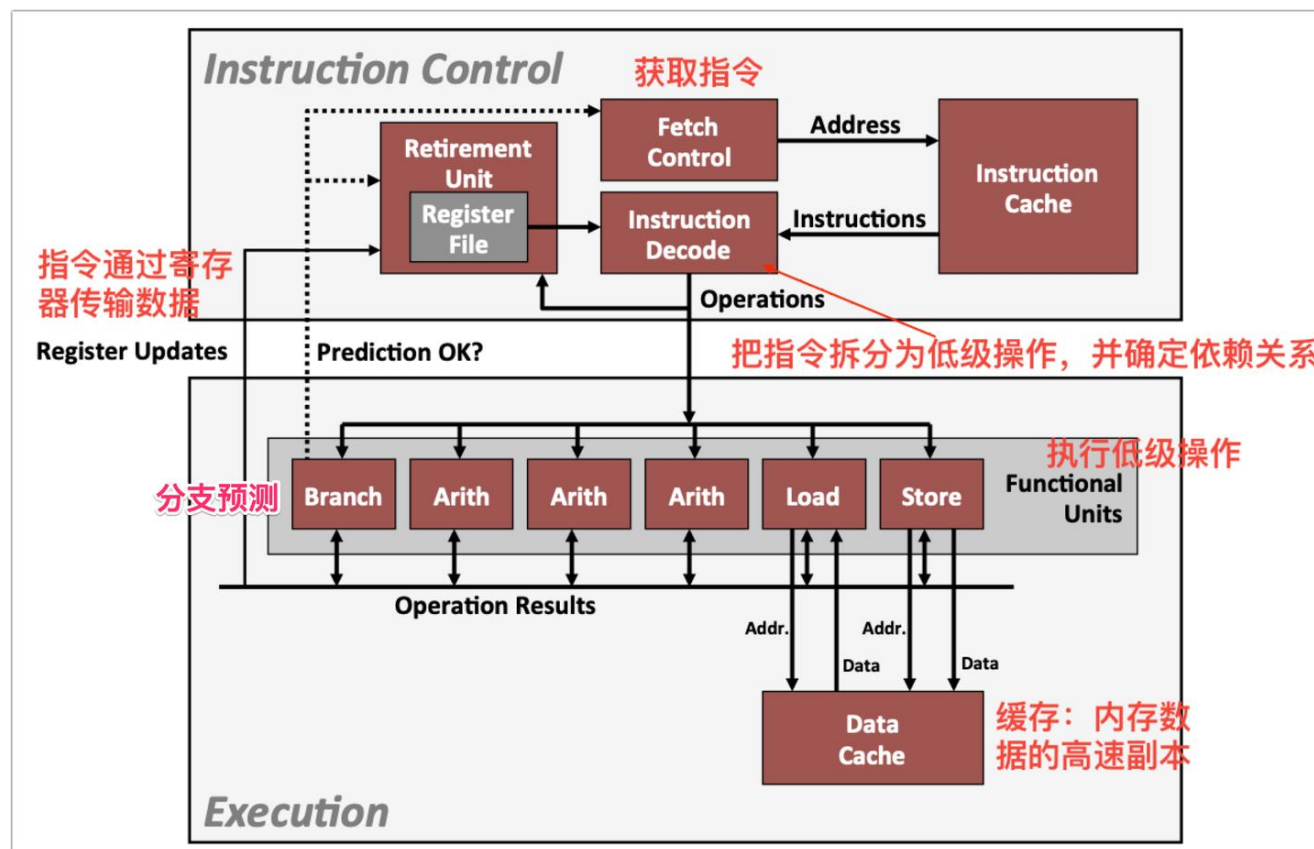
指令控制单元(ICU)

Fetch control

- 包含分支预测的功能

Instruction decode

- 从icache中读取指令，然后翻译为一组微操作
- 例如，`addq %rax, %rdx`转换为单个微操作
- 例如，`addq %rax, 8(%rdx)`转换为内存读取、加法和内存写入三个微操作。



现代处理器架构

■ 执行单元(EU, Execution Unit)

- 接收来自ICU的微操作，分发到各个功能单元执行。

■ Load和Store单元

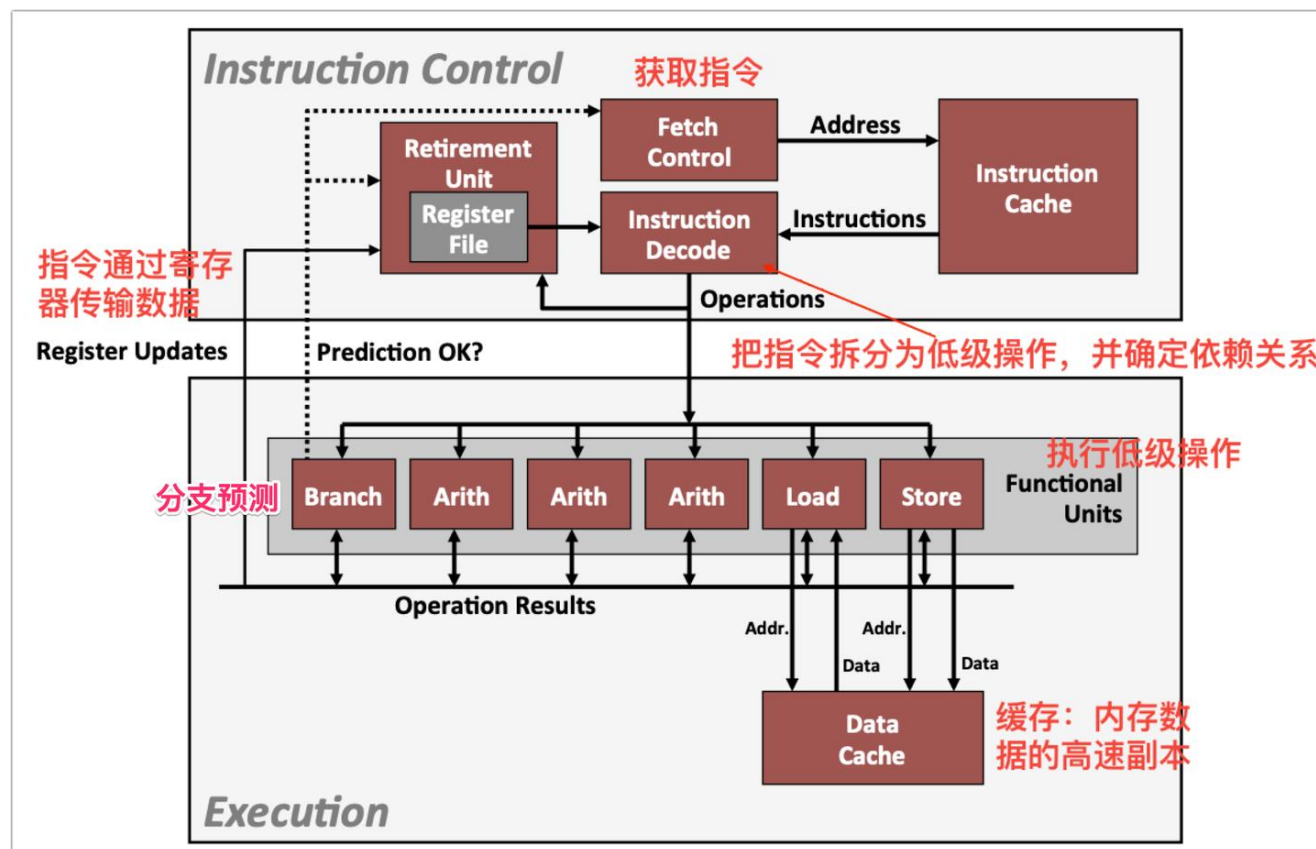
- 包含一个加法器计算地址，和dcache交互

■ Branch单元

- 预测结果会保存在EU内的队列中，若预测错误，则会丢弃保存的执行结果，并通知Fetch Control单元，之后才能获取正确的指令

■ 其它各种功能单元

- 整数运算、浮点乘、整数乘、分支等等



现代处理器架构

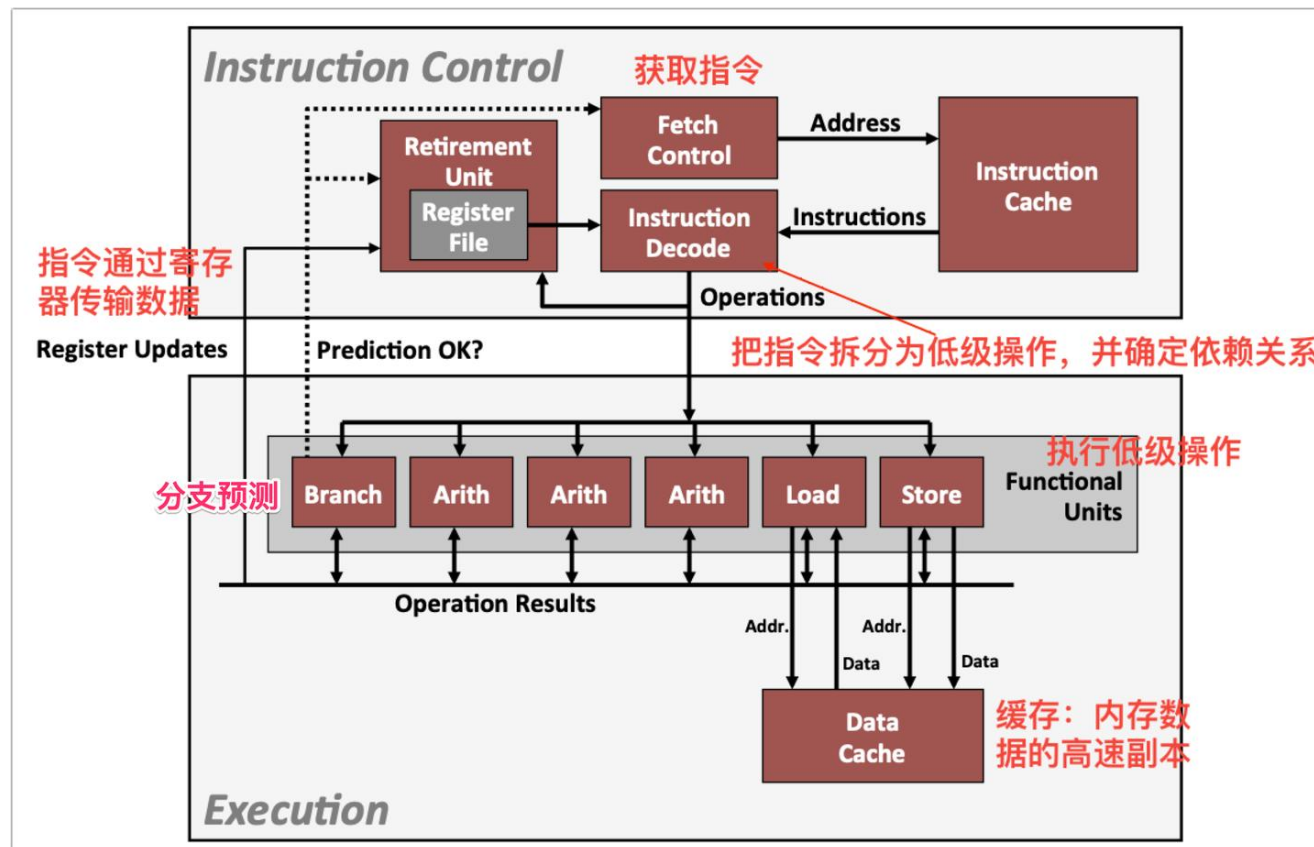
现代处理器架构：乱序 + 超标量



■ 现代处理器一般是乱序且是超标量的。

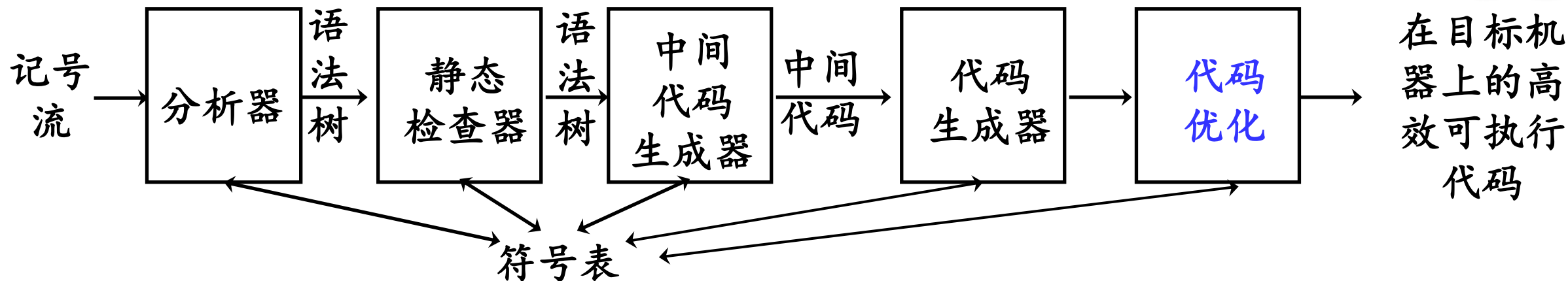
■ 超标量：通过实现多个硬件单元，可以在每个时钟周期执行多个操作

■ 乱序：指令执行的顺序和二进制代码中的顺序不一定相同



现代处理器架构

本节提纲



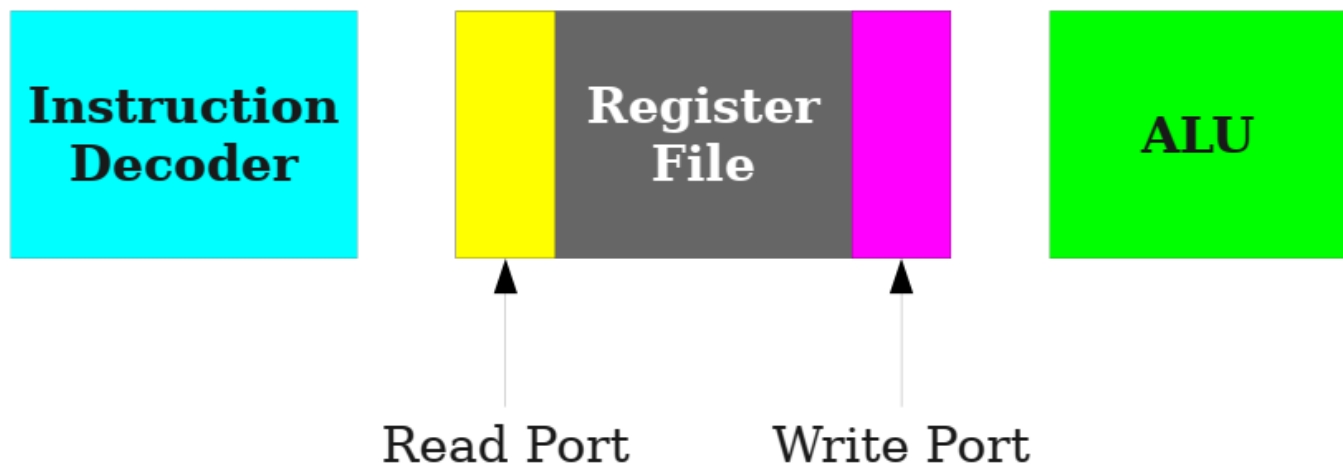
- ❑ 现代处理器架构
- ❑ 流水线并行的例子
- ❑ 指令调度与数据依赖分析
- ❑ 数据依赖指导下的指令调度

add \$t2, \$t0, \$t1 # \$t2 = \$t0 + \$t1

add \$t5, \$t3, \$t4 # \$t5 = \$t3 + \$t4

add \$t8, \$t6, \$t7 # \$t8 = \$t6 + \$t7

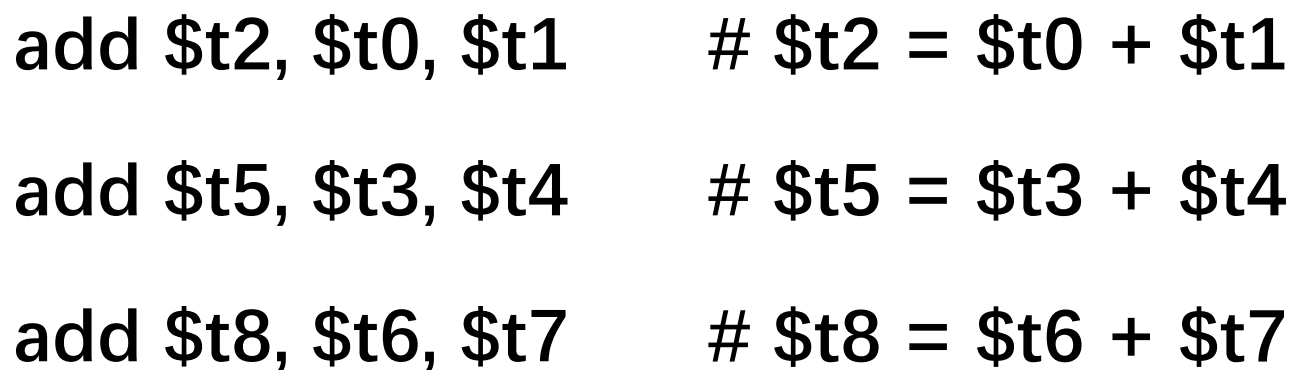
处理器流水线



`add $t2, $t0, $t1` `# $t2 = $t0 + $t1`

`add $t5, $t3, $t4` `# $t5 = $t3 + $t4`

`add $t8, $t6, $t7` `# $t8 = $t6 + $t7`

[illegible]

[illegible]

[illegible]

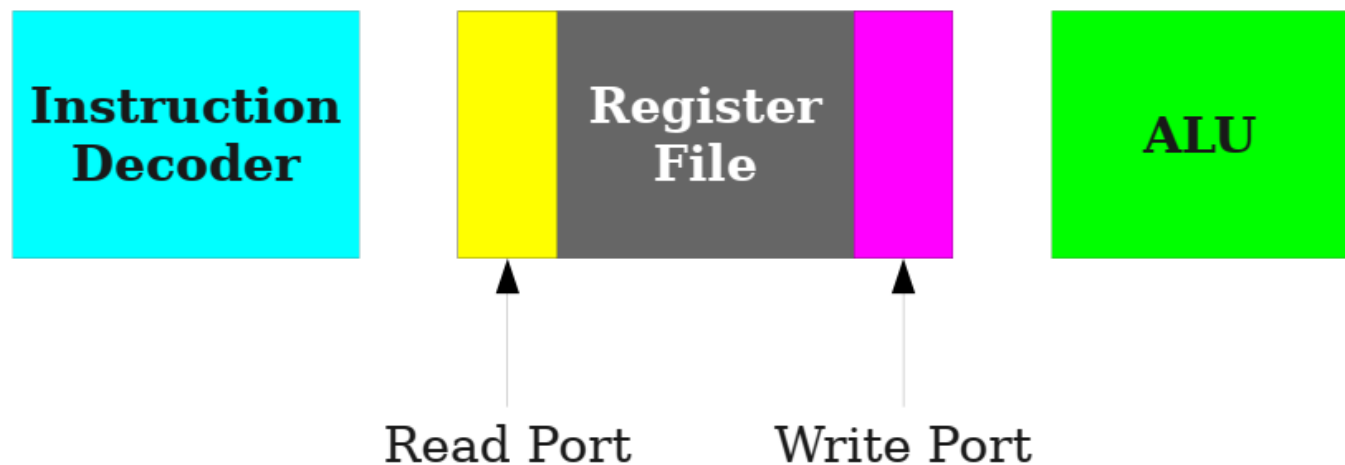
[illegible]



ID	RR	ALU	RW

[illegible]

复杂的流水线并行



add \$t2, \$t0, \$t1 # \$t2 = \$t0 + \$t1

add \$t4, \$t3, \$t2 # \$t4 = \$t3 + \$t2

add \$t7, \$t5, \$t6 # \$t7 = \$t5 + \$t6

add \$t0, \$t0, \$t7 # \$t0 = \$t0 + \$t7

[illegible]



add \$t0, \$t0, \$t7 # \$t0 = \$t0 + \$t7

[illegible]



ID	RR	ALU	RW



ID	RR	ALU	RW

[illegible]



+ \$t1

+ \$t2

add \$t4, \$t3, \$t1

add \$t7, \$t5, \$t6 # \$t7 = \$t5 + \$t6

add \$t0, \$t0, \$t7 # \$t0 = \$t0 + \$t7

ID	RR	ALU	RW

[illegible]

[illegible]

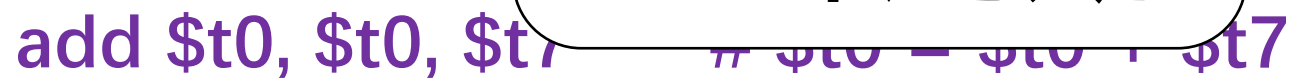
[illegible]

[illegible]



ID	RR	ALU	RW

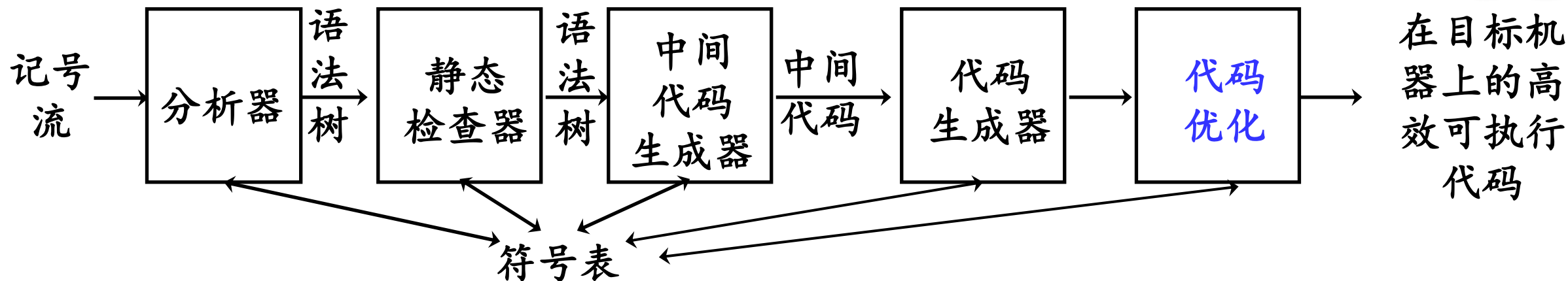
[illegible]



节省两个 时钟周期

[illegible]

本节提纲



- ❑ 现代处理器架构
- ❑ 流水线并行的例子
- ❑ 指令调度与数据依赖分析
- ❑ 数据依赖指导下的指令调度

□优化的起源

- 由于处理器流水线并行机制，指令的执行顺序对性能有较大影响。

□指令调度

- 重排机器代码指令，旨在最小化执行特定指令序列所需的时钟周期数。
- 任意编译器均支持指令调度。

□理论和技术挑战

- 然而，在处理器流水线上执行的顺序代码内含着一些指令之间的依赖关系，在指令调度期间执行的任何转换都必须保留这些依赖关系，以维护被调度代码的逻辑。

□ read-after-write, RAW

- 当一条指令读取另一条指令写入的结果时，会产生写后读相关性，读指令必须在写指令一定时钟周期后再读取而不会产生阻塞。

X = ...
... = X

□ write-after-read, WAR

- 当一条指令写在另一条指令的操作数上时，会产生反向依赖或称读后写依赖。读指令必须在写指令之前经过适当的周期数才能安全读取，而不阻塞写指令。

... = X
X = ...

□ write-after-write, WAW

- 如果两条指令写入同一个目标，就会产生单个输出或写后写依赖关系

X = ...
X = ...

分析数据依赖关系



$$t_0 = t_1 + t_2$$

$$t_1 = t_0 + t_1$$

$$t_3 = t_2 + t_4$$

$$t_0 = t_1 + t_2$$

$$t_5 = t_3 + t_4$$

$$t_6 = t_2 + t_7$$

分析数据依赖关系



$$t0 = t1 + t2$$

$$t1 = t0 + t1$$

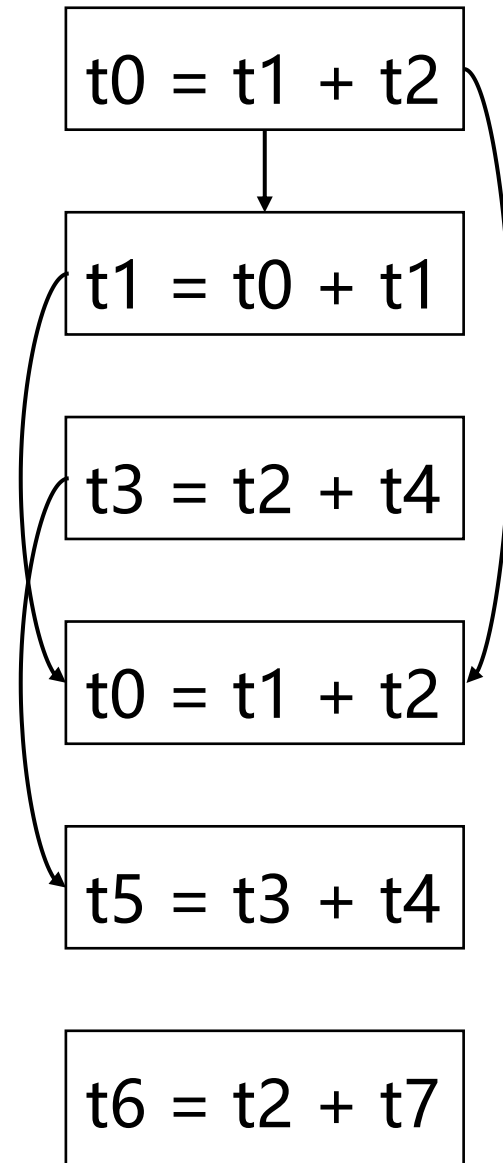
$$t3 = t2 + t4$$

$$t0 = t1 + t2$$

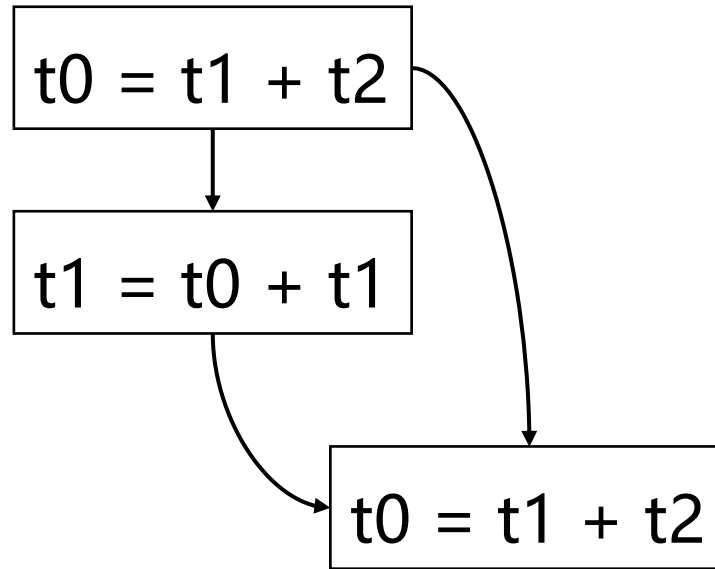
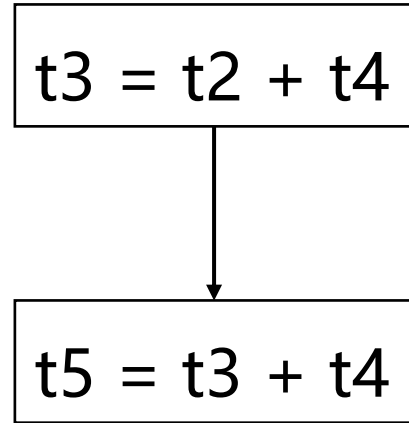
$$t5 = t3 + t4$$

$$t6 = t2 + t7$$

分析数据依赖关系



分析数据依赖关系



$t6 = t2 + t7$

□ 一个基本块的指令数据依赖图：

- 每个节点表示单个机器指令
- 每一条边代表了两条指令间存在数据依赖，否则就没有依赖

□ 依赖图是一个有向无环图，directed acyclic graph (DAG)

- **Directed**: 代表了计算的顺序
- **Acyclic**: 不能存在环状依赖 (why?)

□ 合法的指令调度

- 条件：一条指令不能先于他的祖先节点执行

□ 实现方法

- 对依赖图进行拓扑排序(topological sort)

- John L. Hennessy and Thomas Gross. 1983. **Postpass Code Optimization of Pipeline Constraints**. ACM Trans. Program. Lang. Syst. 5, 3 (July 1983), 422–448. <https://doi.org/10.1145/2166.357217>



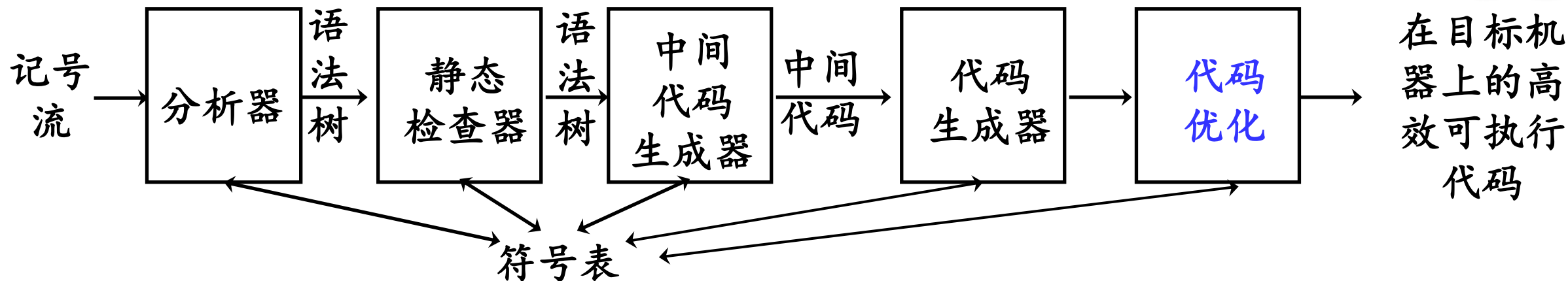
John L. Hennessy



David Patterson

- 2017年，Hennessy和Patterson共同获得图灵奖。
- 获奖演说：
 - A New Golden Age for Computer Architecture: Domain-Specific Hardware/Software Co-Design, Enhanced Security, Open Instruction Sets, and Agile Chip Development

本节提纲



- ❑ 现代处理器架构
- ❑ 流水线并行的例子
- ❑ 指令调度与数据依赖分析
- ❑ 数据依赖指导下的指令调度

数据依赖指导下的指令调度



$$t3 = t2 + t4$$



$$t5 = t3 + t4$$

$$t0 = t1 + t2$$



$$t1 = t0 + t1$$



$$t0 = t1 + t2$$

$$t6 = t2 + t7$$

数据依赖指导下的指令调度



$$t3 = t2 + t4$$



$$t5 = t3 + t4$$

$$t0 = t1 + t2$$



$$t1 = t0 + t1$$



$$t0 = t1 + t2$$

$$t6 = t2 + t7$$

$$t3 = t2 + t4$$

数据依赖指导下的指令调度



$$t3 = t2 + t4$$



$$t5 = t3 + t4$$

$$t0 = t1 + t2$$



$$t1 = t0 + t1$$



$$t0 = t1 + t2$$

$$t6 = t2 + t7$$

$$t3 = t2 + t4$$

数据依赖指导下的指令调度



$$t3 = t2 + t4$$



$$t5 = t3 + t4$$

$$t0 = t1 + t2$$



$$t1 = t0 + t1$$



$$t0 = t1 + t2$$

$$t6 = t2 + t7$$

$$t3 = t2 + t4$$

$$t5 = t3 + t4$$

数据依赖指导下的指令调度



$$t3 = t2 + t4$$



$$t5 = t3 + t4$$

$$t0 = t1 + t2$$



$$t1 = t0 + t1$$

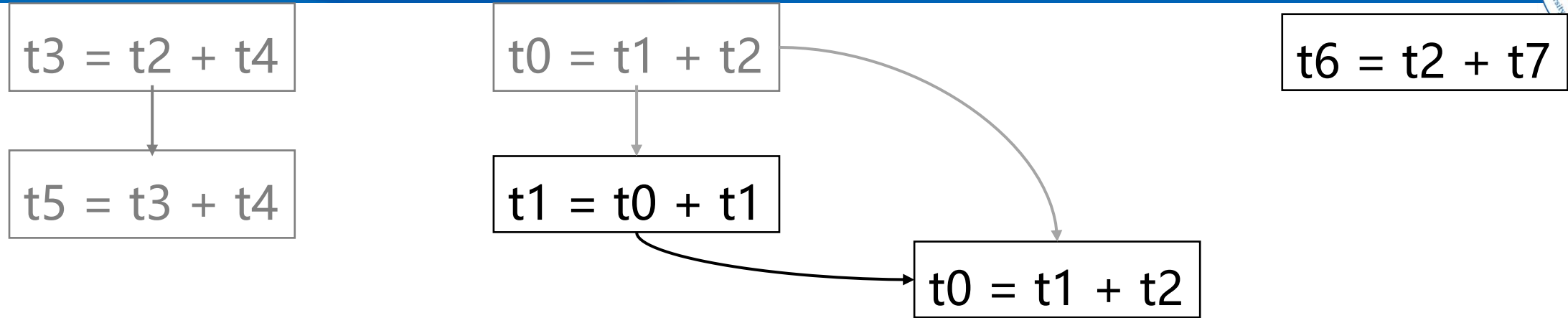


$$t0 = t1 + t2$$

$$t6 = t2 + t7$$

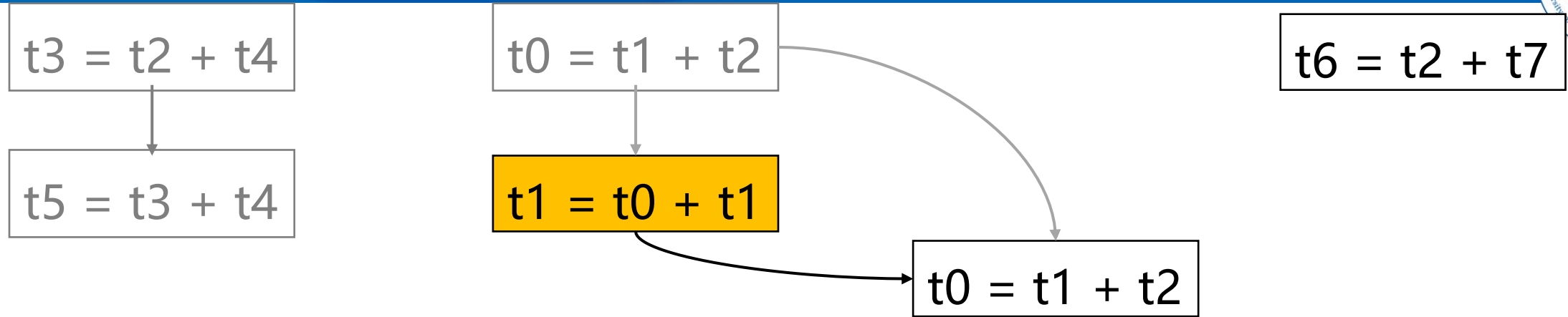
$t3 = t2 + t4$
$t5 = t3 + t4$

数据依赖指导下的指令调度



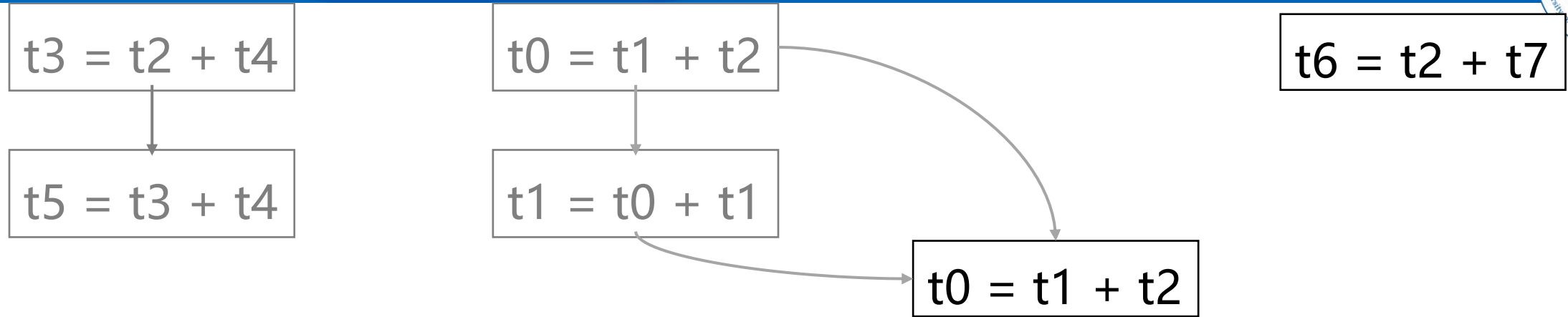
$t3 = t2 + t4$
$t5 = t3 + t4$
$t0 = t1 + t2$

数据依赖指导下的指令调度



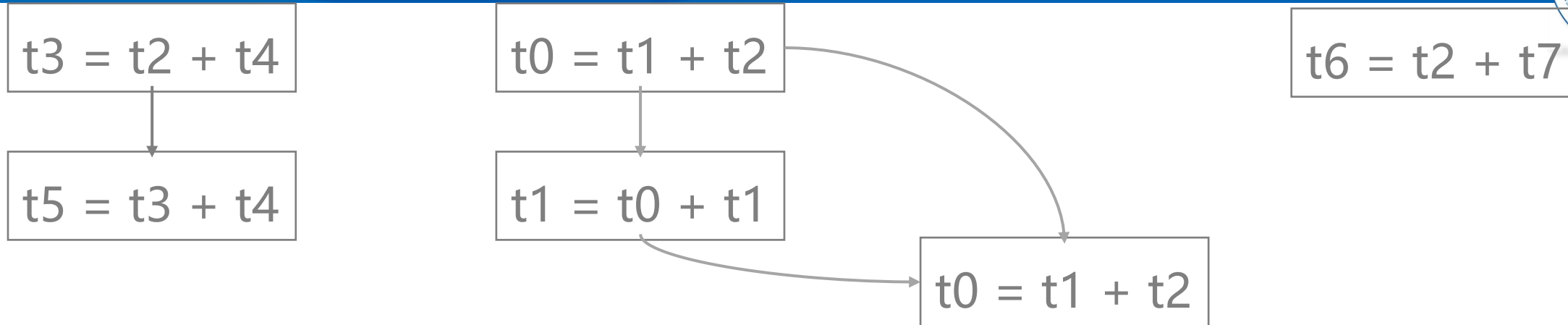
$t3 = t2 + t4$
$t5 = t3 + t4$
$t0 = t1 + t2$

数据依赖指导下的指令调度



$t3 = t2 + t4$
$t5 = t3 + t4$
$t0 = t1 + t2$
$t1 = t0 + t1$

数据依赖指导下的指令调度



$t3 = t2 + t4$
$t5 = t3 + t4$
$t0 = t1 + t2$
$t1 = t0 + t1$
$t0 = t1 + t2$
$t6 = t2 + t7$

数据依赖指导下的指令调度



$$t3 = t2 + t4$$



$$t5 = t3 + t4$$

$$t0 = t1 + t2$$



$$t1 = t0 + t1$$



$$t0 = t1 + t2$$

$$t6 = t2 + t7$$

$t3 = t2 + t4$
$t5 = t3 + t4$
$t0 = t1 + t2$
$t1 = t0 + t1$
$t0 = t1 + t2$
$t6 = t2 + t7$

数据依赖指导下的指令调度



$$t3 = t2 + t4$$



$$t5 = t3 + t4$$

$$t0 = t1 + t2$$



$$t1 = t0 + t1$$



$$t0 = t1 + t2$$

$$t6 = t2 + t7$$

$t3 = t2 + t4$
$t5 = t3 + t4$
$t0 = t1 + t2$
$t1 = t0 + t1$
$t0 = t1 + t2$
$t6 = t2 + t7$

$t0 = t1 + t2$
$t3 = t2 + t4$
$t6 = t2 + t7$
$t1 = t0 + t1$
$t5 = t3 + t4$
$t0 = t1 + t2$

❑ 数据依赖图可能有许多有效的拓扑排序。

■ 该如何选择一种能与流水线完美配合的排序方式呢？

❑ 寻找最快的指令时间表是众所周知的 NP 难题。

■ 不要指望很快就能找到多项式时间算法！

❑ 在实践中使用启发式方法

1. 将可以不受干扰地运行完成的指令安排在会造成干扰的指令之前。
2. 将依赖关系较多的指令安排在依赖关系较少的指令之前。
3. 对 **DAG** 进行加权调整！（边的权重为指令等待时间）

升级版的指令调度



$t3 = t2 + t4$



$t5 = t3 + t4$

$t6 = t2 + t7$

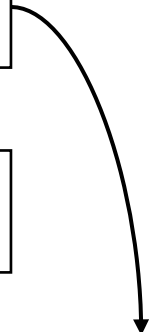
$t0 = t1 + t2$



$t1 = t0 + t1$



$t0 = t1 + t2$



升级版的指令调度



0	$t3 = t2 + t4$
---	----------------



0	$t5 = t3 + t4$
---	----------------

0	$t6 = t2 + t7$
---	----------------

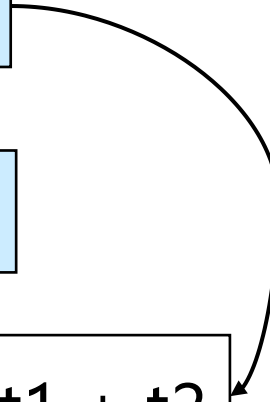
$t0 = t1 + t2$	0
----------------	---



$t1 = t0 + t1$	0
----------------	---



0	$t0 = t1 + t2$
---	----------------



升级版的指令调度



0	$t3 = t2 + t4$
---	----------------

+3

0	$t5 = t3 + t4$
---	----------------

0	$t6 = t2 + t7$
---	----------------

$t0 = t1 + t2$	0
----------------	---

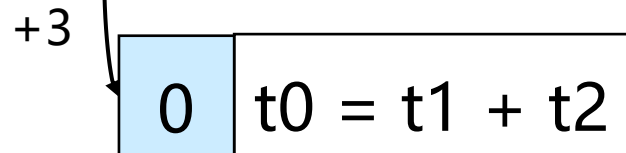
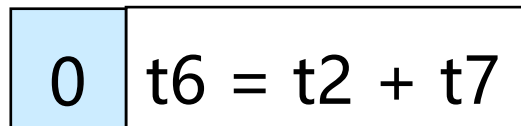
+3

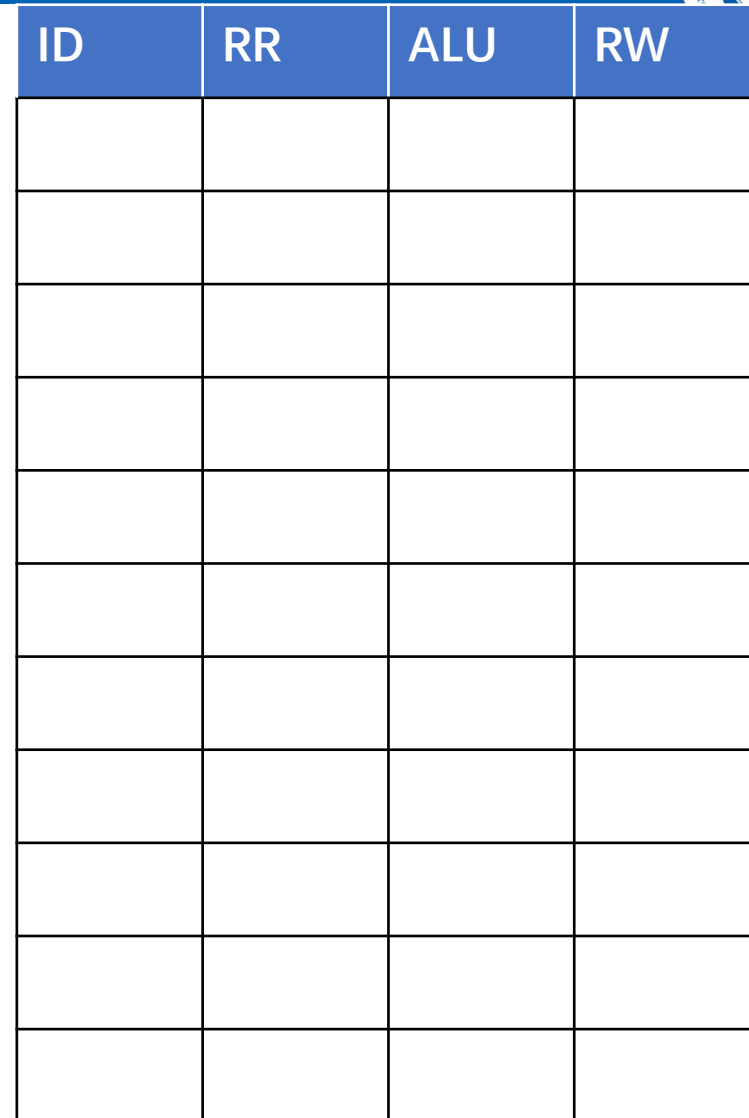
$t1 = t0 + t1$	0
----------------	---

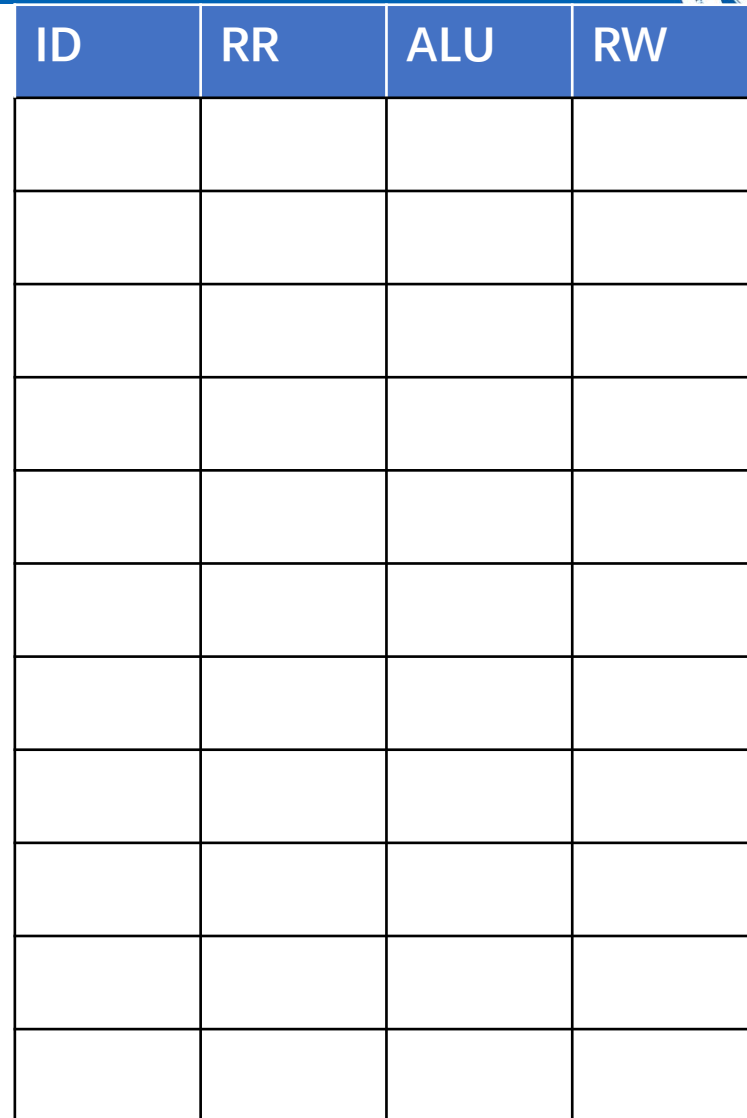
+3

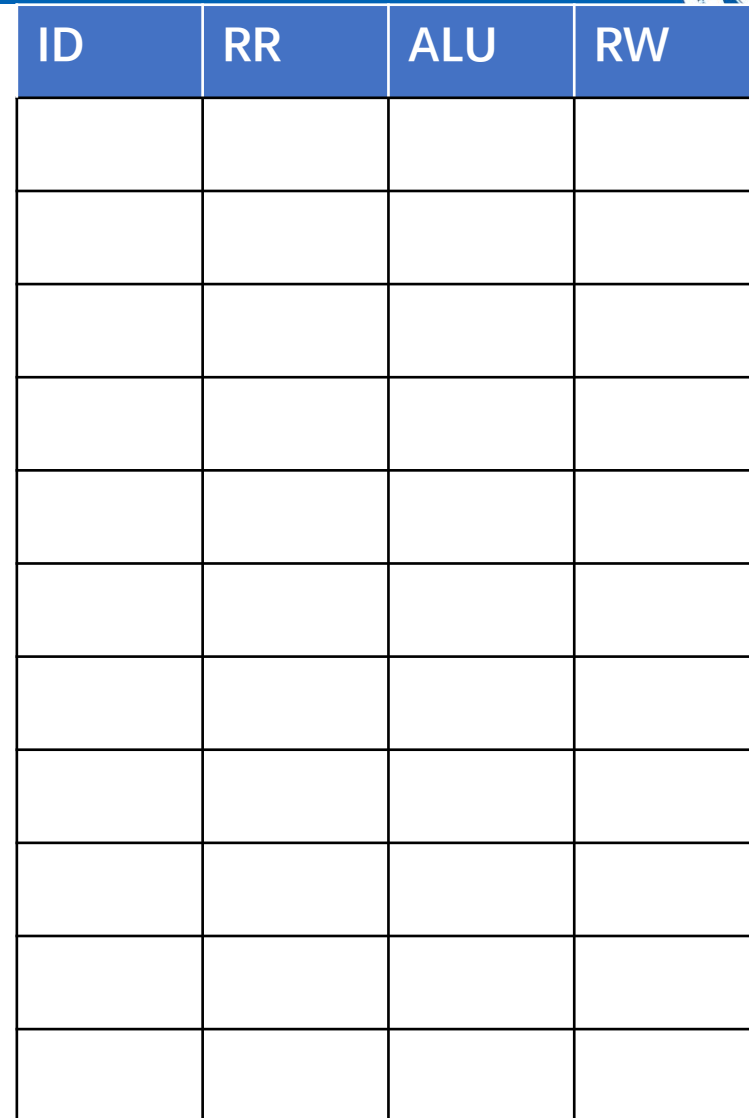
0	$t0 = t1 + t2$
---	----------------

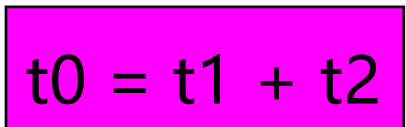
+3

[illegible]







[illegible]

升级版的指令调度



0

$t3 = t2 + t4$

+3

0

$t5 = t3 + t4$

0

$t6 = t2 + t7$

$t0 = t1 + t2$

$t1 = t0 + t1$

3

+3

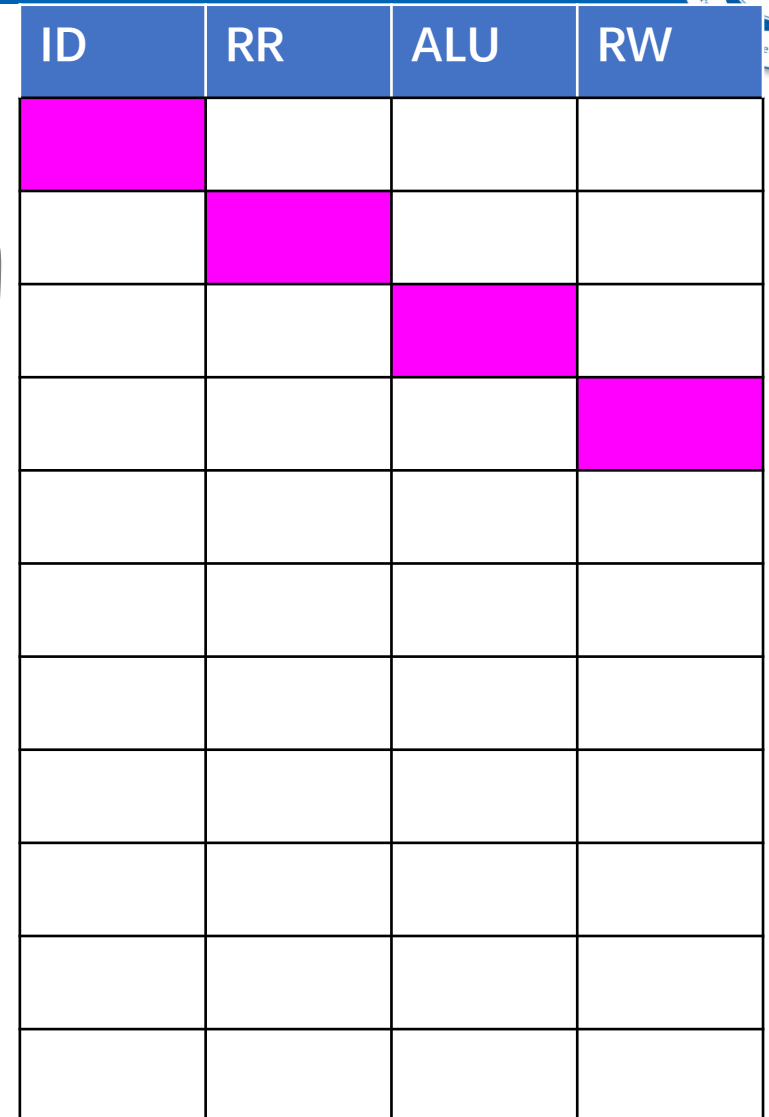
3

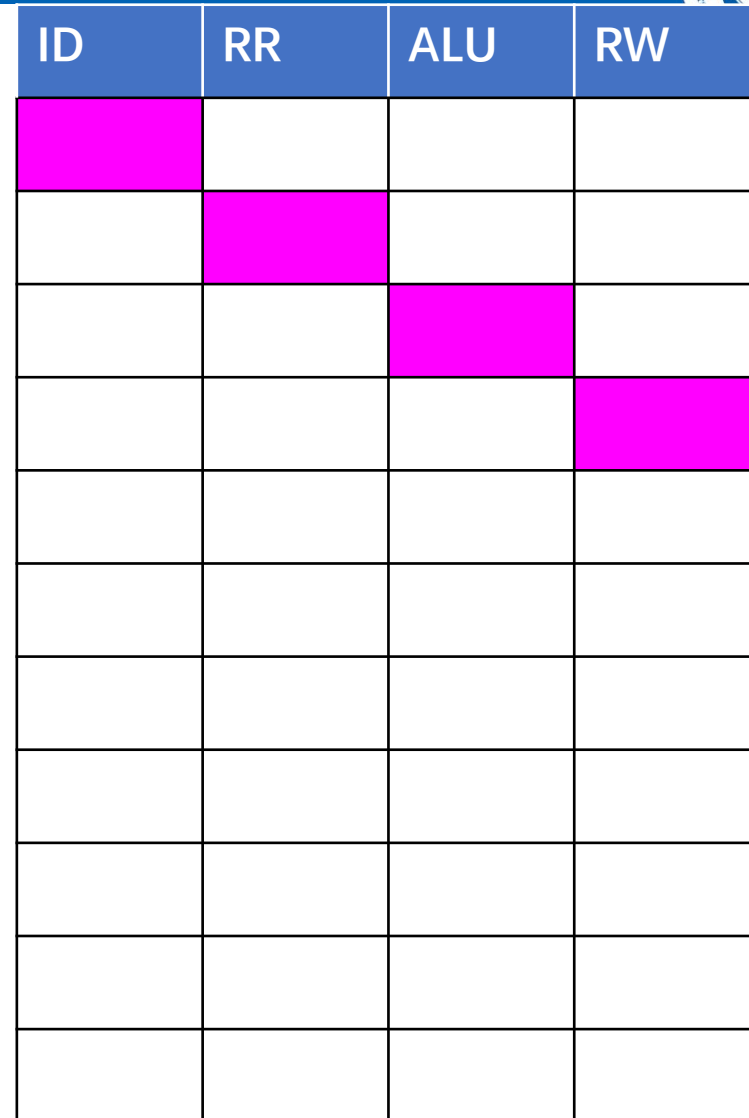
$t0 = t1 + t2$

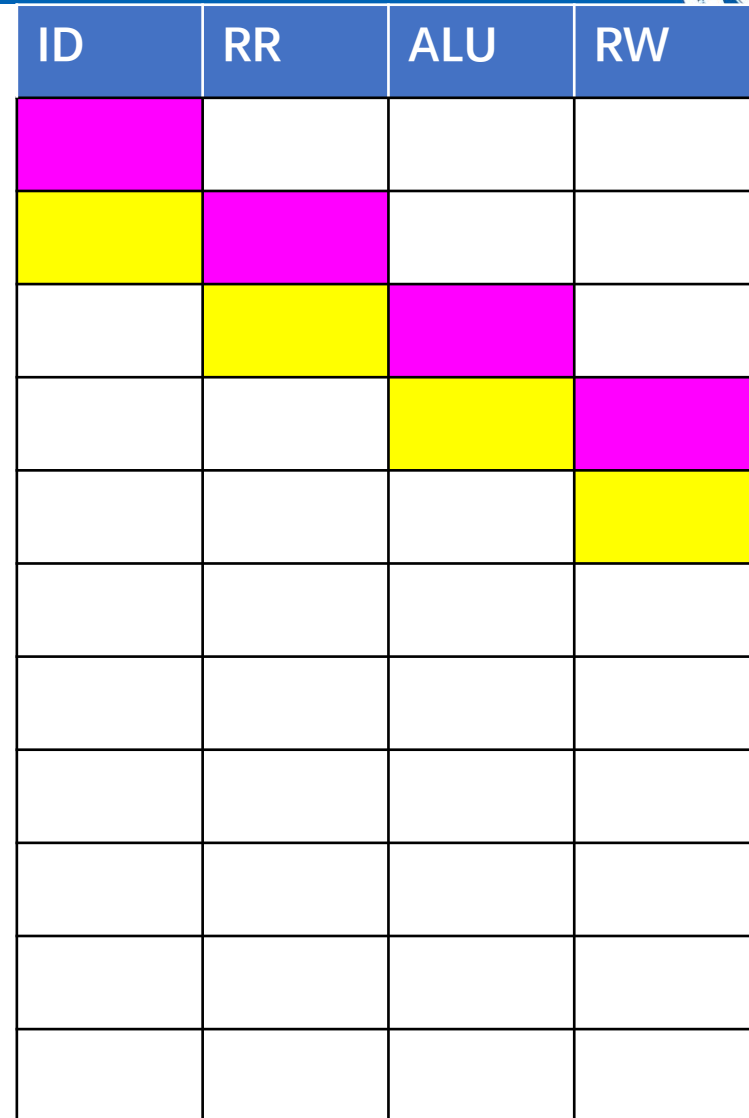
ID	RR	ALU	RW

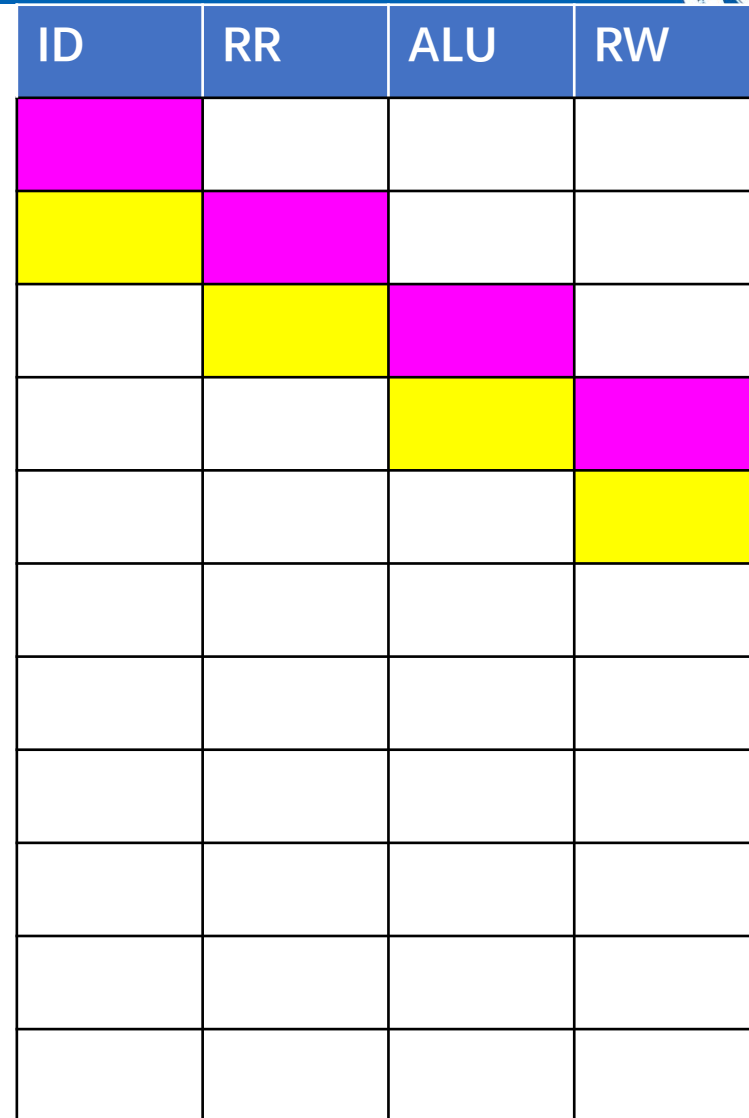
$t0 = t1 + t2$

$t3 = t2 + t4$


$$t_3 = t_2 + t_4$$


$$t_3 = t_2 + t_4$$





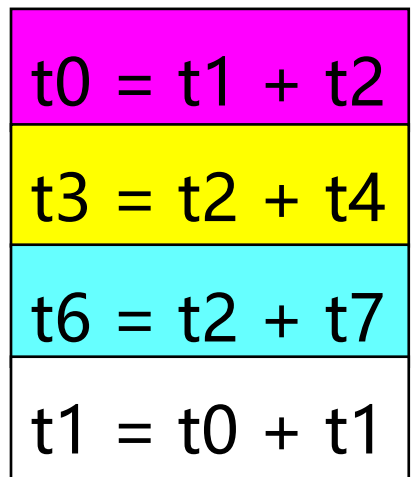


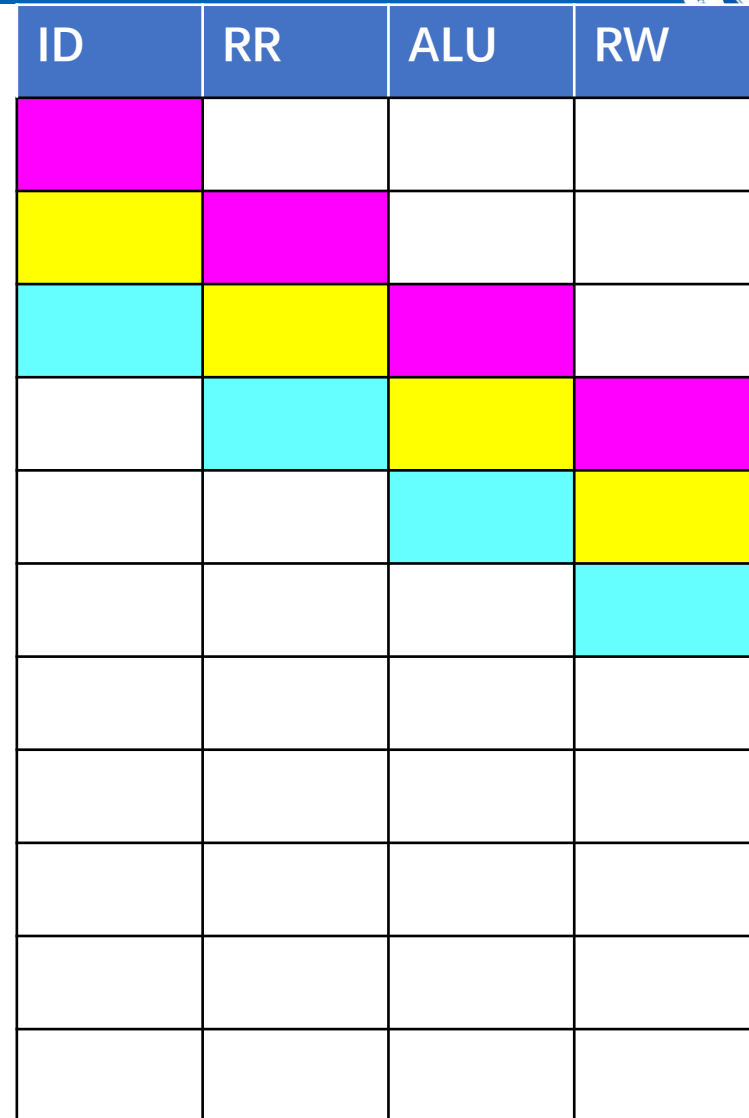
4 | $t_5 = t_3 + t_4$

$$0 \mid t_6 = t_2 + t_7$$
$$t_0 = t_1 + t_2$$
$$t_1 = t_0 + t_1 \quad | \quad 3$$

+3

3 $t_0 = t_1 + t_2$
$$t_0 = t_1 + t_2$$
$$t_3 = t_2 + t_4$$
$$t_6 = t_2 + t_7$$

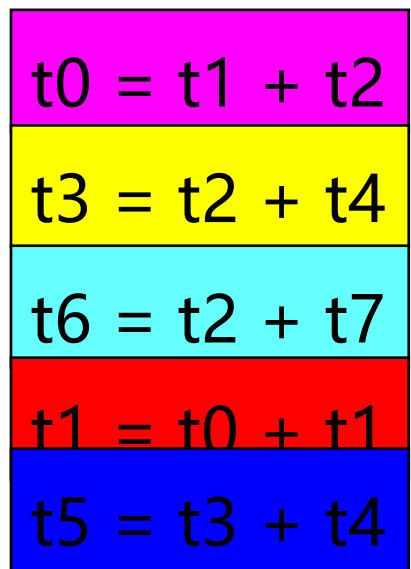
[illegible]

[illegible]

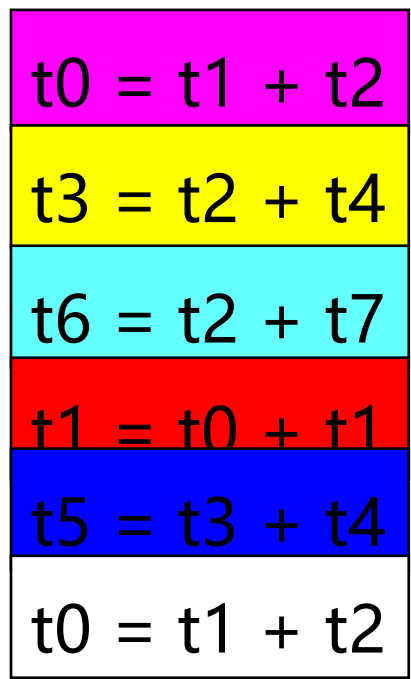
[illegible]

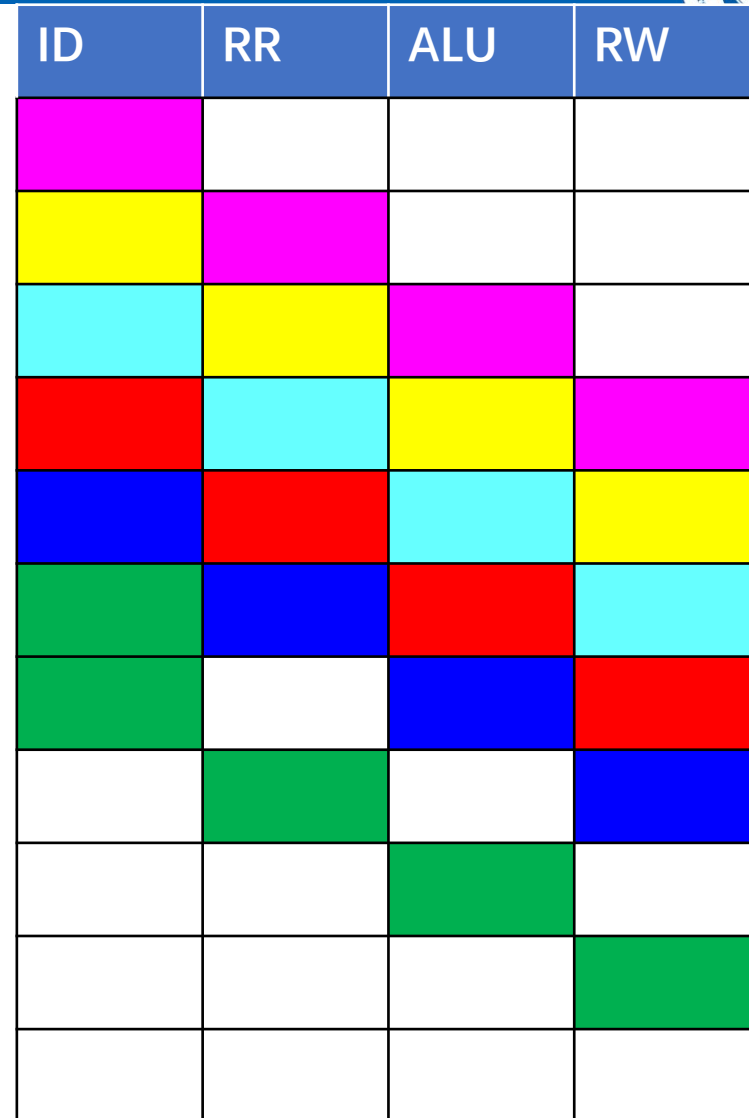


$t_0 = t_1 + t_2$
$t_3 = t_2 + t_4$
$t_6 = t_2 + t_7$
$t_1 = t_0 + t_1$
$t_5 = t_3 + t_4$



ID	RR	ALU	RW

[illegible]



$t_0 = t_1 + t_2$
$t_3 = t_2 + t_4$
$t_6 = t_2 + t_7$
$t_1 = t_0 + t_1$
$t_5 = t_3 + t_4$
$t_0 = t_1 + t_2$

[illegible]

- 现代优化编译器可以进行更积极的调度，以获得巨大的性能提升。
- 一种强大的技术：循环展开(loop unrolling)
 - 一次展开多个循环迭代。
 - 使用前面介绍的调度算法更智能地调度指令。
 - 可以在循环迭代中找到流水线级并行性。

软件流水线(Software pipeline)

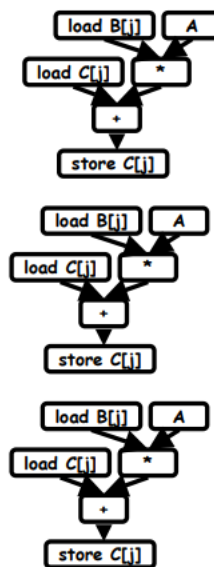


- 通过并行执行来自不同循环体的指令来加快循环程序的执行速度;
- 在前一个循环体未结束前启动下一个新的循环体,来达成循环体时间上的并行性;
- 相比于简单的展开循环,软件流水线在优化资源使用的同时保持代码的简洁。

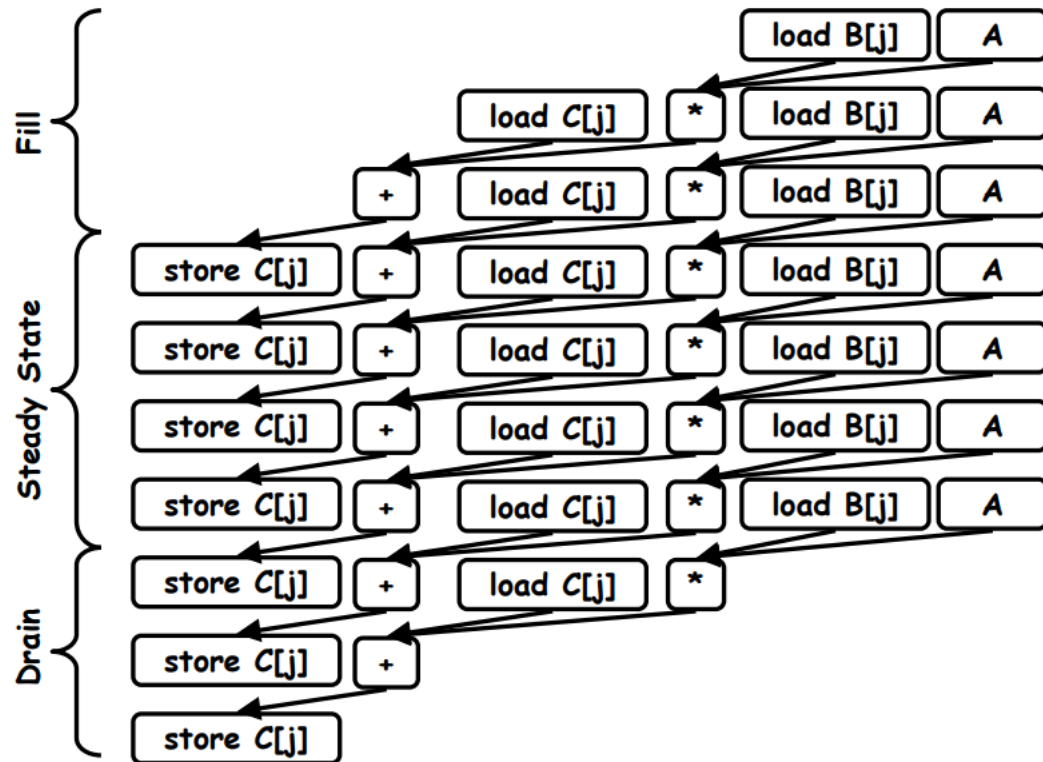
```
for(j = 0; j < MAX; j++)
```

```
  C[j] += A * B[j];
```

Not pipelined:



Pipelined:





一起努力 打造国产基础系统软件体系！

李 诚

国家高性能计算中心(合肥)、信息与计算机国家级实验教学示范中心

计算机科学与技术学院

2025年11月17日